Kathara Lab

Kathara is a **network emulation tool**. Within the Kathara environment each network node is implemented by a docker container, and each interconnection link is emulated by using a virtual network. Each device can be configured to have an arbitrary number of (virtual) ethernet network interfaces. Each node can be seen as an independent linux machine, for you to configure from scratch.

Create a Kathara Laboraty

First of all, we must create a configuration file.

```
francesco@PCFrancesco:~/Desktop$ mkdir lab_test
francesco@PCFrancesco:~/Desktop$ cd lab_test
francesco@PCFrancesco:~/Desktop/lab_test$ nano lab.conf
```

Here we specify three things:

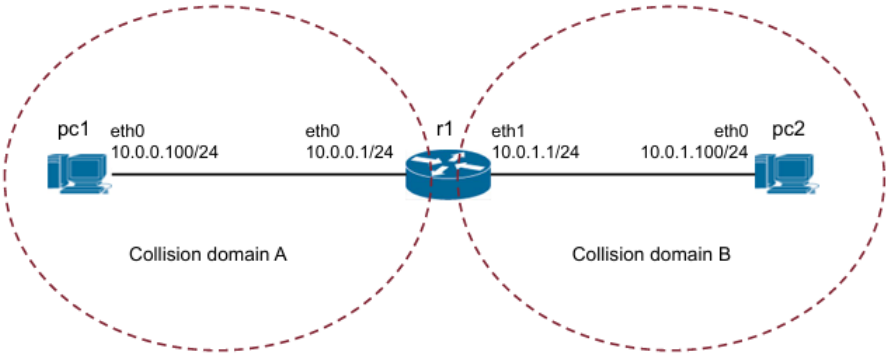
- **node names** (must be unique)
- **network interfaces** of the node, in the form of eth0, eth1, …
- **collision domains** of the interfaces

```
pc1[0]=A #set eth0 of pc1 to collision domain A
r1[0]=A  #set eth0 of r1 to collision domain A

r1[1]=B  #set eth1 of r1 to collision domain B
pc2[0]=B #set eth0 of pc2 to collision domain B
```

NOTE: There can not be spaces " "

The collision domain is a sort of LAN. All the nodes in the same collision domain will receive the same packets.



Now, we've correctly configured kithara. We can start configuring the network using the command:

- **kathara lstart**: to start the laboratory configured in the working directory
- **kathara lclean**: to stop the laboratory

Once started the laboratory kathara will open a command prompt for each node configured in the configuration file.

Commands

Let's see the most common commands.

Ip link

The ip link command let us show and manage (setting up and down) network interfaces:

```
ip link show          # shows the details of every available iface
ip link set eth0 up   # brings up eth0 interface
ip link set eth0 down  # brings down eth0 iface
```

NOTE: By default, the interfaces are set up (s we don't need to set them up on every new lab)

Ip address

Now that the interfaces are up, we are can assign IPv4 addresses to them. The command to assigning IPv4 addresses:

```
ip address show  # show information about addresses
                 # assigned to interfaces

ip address add a.b.c.d/m dev ethX
# assigns a.b.c.d address with
# submask m to device ethX
```

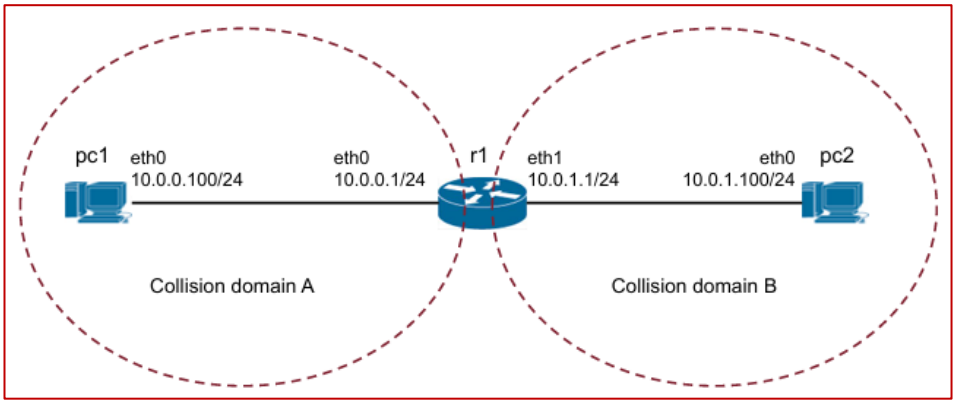
Ping

We can test connection between two hosts using the ping command:

```
ping a.b.c.d      # sends ICMP packets to a.b.c.d
                  # waits for reply
```

It gives us information about Round Trip Time and Time To Live (useful for traceroute) for a connection between 2 hosts.

Exercise: Configure kathara (via *lab.*conf) and use *ip address add* to configure the following network. Then use *ping* from pc1 to r1 and from pc2 to r1:



Solution:

```
francesco@PCFrancesco:~/Desktop$ mkdir lab_test
francesco@PCFrancesco:~/Desktop$ cd lab_test
francesco@PCFrancesco:~/Desktop/lab_test$ nano lab.conf
```

```
pc1[0]=A #set eth0 of pc1 to collision domain A
r1[0]=A  #set eth0 of r1 to collision domain A

r1[1]=B  #set eth1 of r1 to collision domain B
pc2[0]=B #set eth0 of pc2 to collision domain B
```

```
(base) PS C:\Users\39324\Desktop\lab_ex1> kathara lstart
                    Starting Network Scenario
[Deploying collision domains]  2/2
[Deploying devices]            3/3
```

Network Configuration:

```
root@pc1:/# ip address add 10.0.0.100/24 dev eth0
root@pc1:/#
```

```
root@r1:/# ip address add 10.0.0.1/24 dev eth0
root@r1:/# ip address add 10.0.1.1/24 dev eth1
root@r1:/#
```

```
root@pc2:/# ip address add 10.0.1.100/24 dev eth0
root@pc2:/#
```

Ping pc1 -> r1:

```
root@pc1:/# ping 10.0.0.1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=1.19 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.635 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=1.16 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.745 ms
```

Ping pc2 -> r1:

```
PING 10.0.1.1 (10.0.1.1) 56(84) bytes of data.
64 bytes from 10.0.1.1: icmp_seq=1 ttl=64 time=1.10 ms
64 bytes from 10.0.1.1: icmp_seq=2 ttl=64 time=0.976 ms
64 bytes from 10.0.1.1: icmp_seq=3 ttl=64 time=0.831 ms
64 bytes from 10.0.1.1: icmp_seq=4 ttl=64 time=0.362 ms
```

NOTE: If we try to ping pc1 to pc2 we'll get an error.

```
root@pc1:/# ping 10.0.1.100
connect: Network is unreachable
```

This happens because pc1 and pc2 are in two different collision domains and they don't know how to reach each other. To solve this problem, we must use *ip route add* (see below)

Static routes

Let *pc1* and *pc2* be two different nodes on different collision domains. If we try to ping pc1 → pc2, we get "**network unreachable**". But, why? Because pc1 does not know how to reach pc2, because he only has the route to his subnet (collision domain).

Solution: In order to let pc1 and pc2 communicate, we have two ways:

- **Define the "default" route (gateway)**
- **Specify how to reach pc2 subnet from pc1 and vice versa**

The difference between the two approaches will be clearer on bigger topology, but basically by defining a gateway we tell to the host "Give all the packets that are not for your subnet to the gateway, he knows how to route them". In our case the gateway is r1.

With the second approach, we tell to the host "Give all the packets that are for a specific subnet to a specific host (r1), he knows how to route them"

Ip route

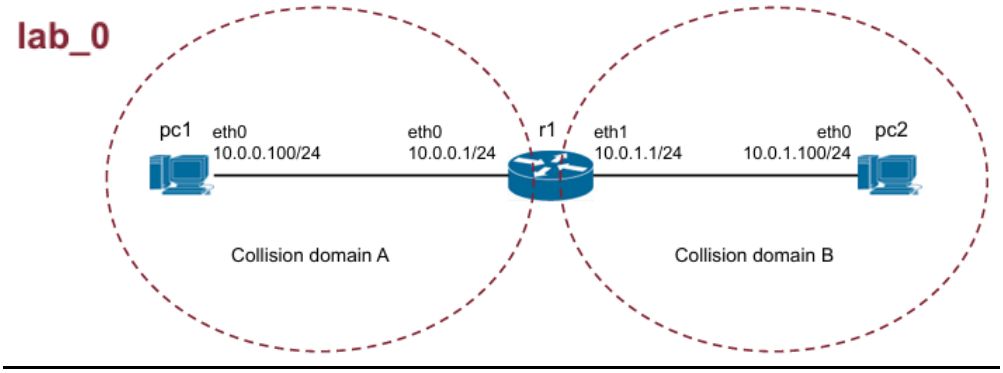
In order to manage routes, we use the command:

```
ip route show    # show the available routes on the host

ip route add a.b.c.d/m via next_hop_ip
# adds the route to subnet a.b.c.d/m via the next_hop_ip

ip route add default via next_hop_ip
ip route add 0.0.0.0/0 via next_hop_ip
# adds the default gateway via the next_hop_ip
```

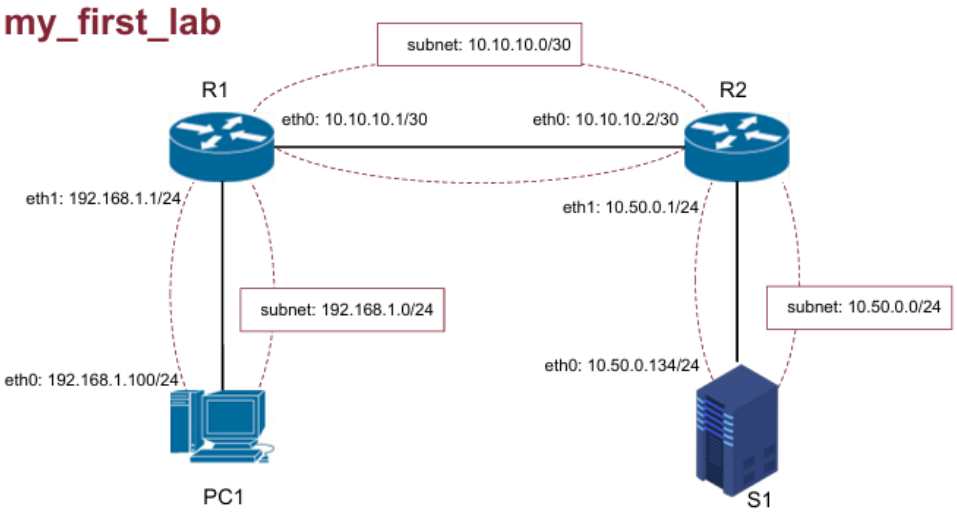
Exercise: Add the route to the previous network that allows pc1 to PING pc2 (and vice versa):



Solution: We should use *ip route add* to tell *pc1*to forward its packets to r1 (then he will know how to send them to *pc2*). Then, we have to use *ip route add* to tell *pc2* how to reach *pc1* (also using r1, then he will know how to send the packets to *pc1*).

[Photo]

Exercise



Make every node able to reach (ping) all other nodes:

1. Create and launch a kathara lab having the topology (collision domains) as in the previous slide
2. Assign static ip addresses as shown via ip address commands
3. Assign necessary routes via ip route commands
4. Ping everybody

Solution: You can find the solved exercise in *Kathara-Lab/Exercises/2-my_first_lab*

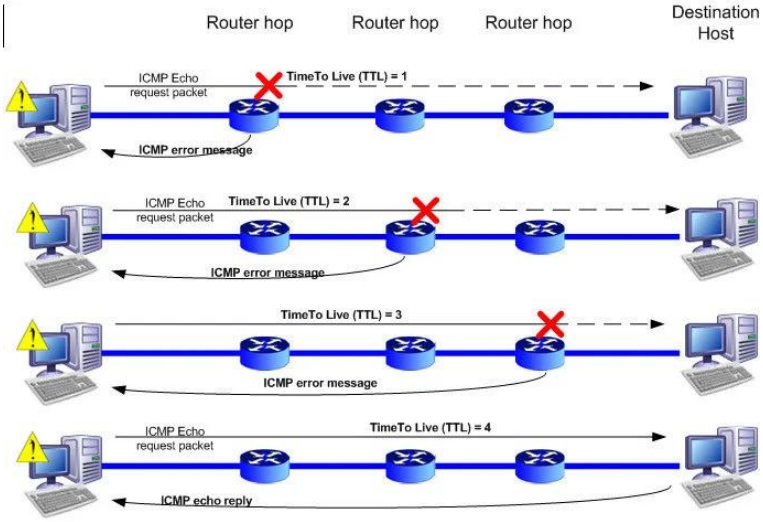
traceroute

Another useful way of testing connection between two hosts is to traceroute the connection via the traceroute command:

```
traceroute a.b.c.d    # finds the number and addresses
                      # of hops to reach a.b.c.d
```

This command exploits the fact that every packet has a **TTL (Time to Live)** field which is X when the packet leaves the interface. Every hop decrease by one the TTL field. When the TTL reaches 0, the host that expired the TTL sends back to the sender an ICMP error packet.

The packet contains the IP address of the host who expired the TTL. By sending packets with TTL starting from 1, traceroute can discover the path taken from the packets to reach a particular destination.



netcat

Netcat allows us to send data over TCP and UDP protocols. In order to use that, we must specify the protocol we want to use, the port we want to use and if we want to send or "listen" on that particular port:

```
nc ip_addr port_num   # connects via TCP to ip_addr
                      # trough port_num via TCP

nc -u ip_addr port_num    # Send UDP datagrams
                          # to ip_addr trough port_num

nc -l -p port_num         # listen for TCP connections on port_num

nc -u -l -p port_num      # listen for UDP datagrams on port_num
```

tcpdump

Linux provides us tools to generate, capture and inspect packets. We can create TCP/UDP packets via the *nc (netcat) command*.

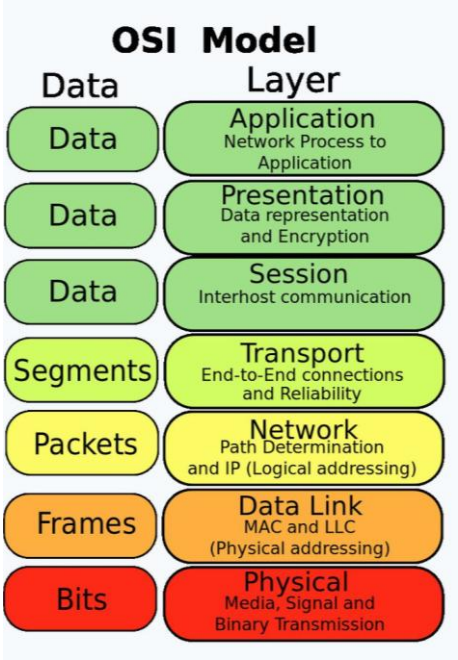
These packets generated can be "captured" via *tcpdump* command and stored to a **"pcap" (packet capture) file.** This file can be later open to be graphically inspected with the **wireshark tool**.

tcpdump allows us to capture packets on a physical interface.

```
tcpdump -i iface_name -w filename.pcap
# capture packets on iface_name and save captured packets on filename.pcap
```

OSI Stack Protocol

The Open Systems Interconnection (OSI) model is the universal framework for computer networking. It breaks down the complex process of transmitting data across a network into distinct, manageable layers.



The Physical Layer (L1): is responsible to define how information is transmitted physically (copper wire, fiber optics, air, etc.) between two entities.

The Data Link Layer (L2): manages, among other things, physical addressing between adjacent nodes in the network. Each network interface has assigned a unique MAC address, used as id. (A single device can have multiple network interfaces!! Each of them has a unique MAC address in the world)

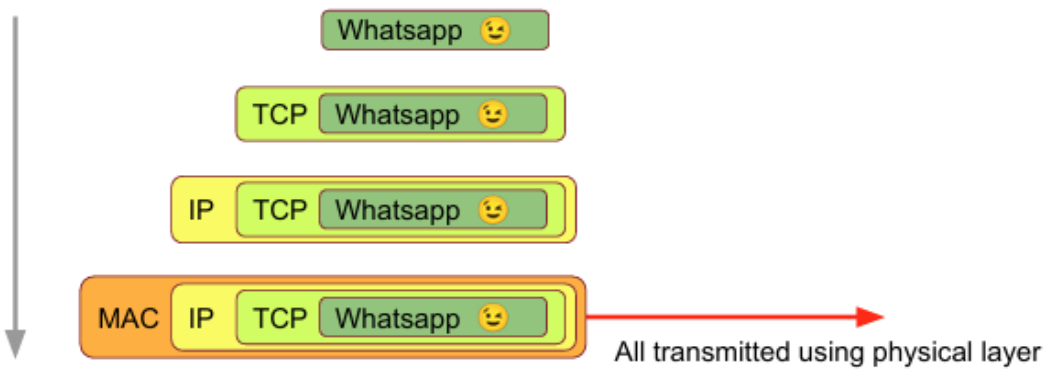
The Network Layer (L3): manages addressing between distant nodes (e.g., nodes in different subnets). Each node has a unique IP address in the subnet.

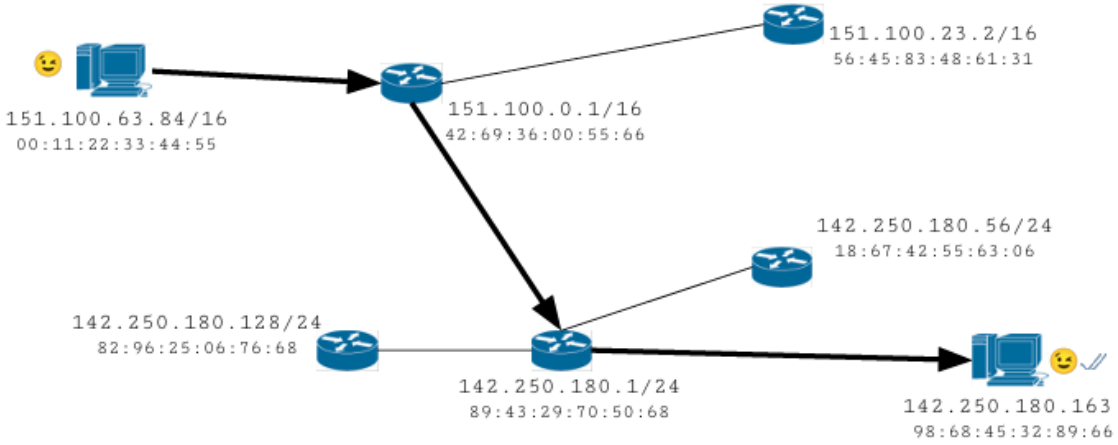
The Transport Layer (L4): manages the end-to-end transport of data (e.g., add ports, multiplex the connections, adds packet acknowledgments, congestion control, prevent out of order packets, etc…). Every well-known application protocol (HTTPS, MQTT, …) runs on top of the transport layer. Netcat uses L4 protocols (TCP or UDP) to send arbitrary data between two hosts.

The Application Layer(s): Application protocols (HTTP, FTP, MQTT) transmit user data end-to-end. In between Transport and Application protocols, there could be protocols such as TLS/SSL to secure application packets.

Packet Encapsulation

When sending a packet using a certain protocol in the stack layer, it gets encapsulated using all the lower layers (and then de-encapsulated at the receiver).





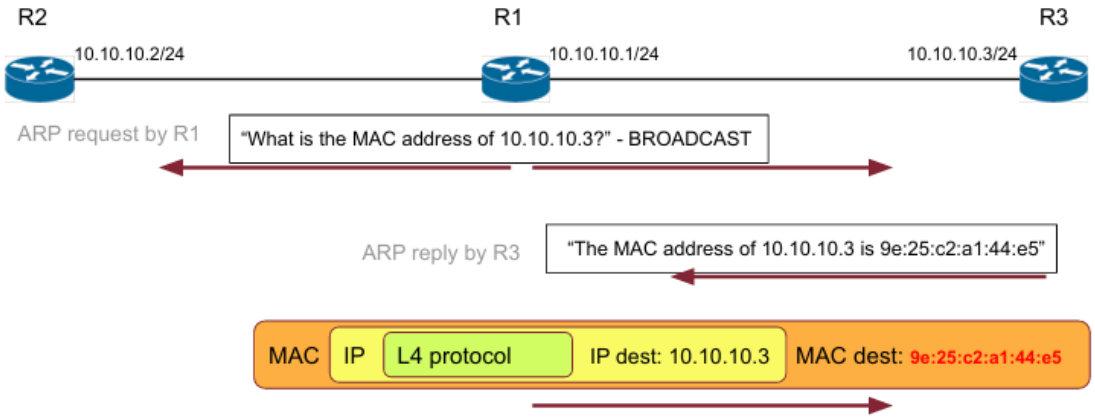
Each node has (at least) one IP address (L3) and one MAC address (L2). But, why?

- **MAC Addresses** uniquely identify a network interface. Each manufacturer manages its own pool of addresses1, and assigns one of them to each network interface. Constant throughout all the lifespan of the device. MAC addresses are used in physical addressing to uniquely identify a node in my neighbors
- **IP addresses** are assigned logically, in such a way to simplify the routing process. Each address belongs to a pool of addresses (subnet). May change over time. IP addresses and subnetting enable routing

In the previous slide, we routed packets from 151.100.63.84 to 142.250.180.163 It required 3 physical addressing, one for each "hop" in the routing.

ARP Basics

The Address Resolution Protocol (ARP) is used to discover the link layer MAC address associated to a given internet layer IP address. Applications are required to specify the destination IP address to which send/receive data, they know nothing about MAC addresses.

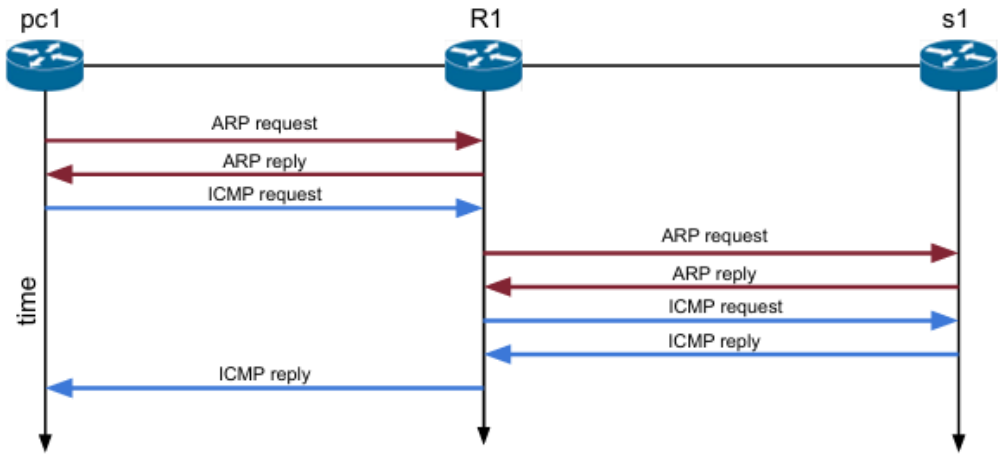


The MAC address collected by ARP are stored in a cache, accessible with *ip neigh* command

```
192.168.1.1 dev wlp1s0 lladdr 14:14:50:7a:d0:bb DELAY
192.168.1.7 dev wlp1s0 lladdr 1e:80:eb:d2:28:e8 REACHABLE
192.168.1.250 dev wlp1s0 lladdr e4:4f:01:00:31:08 REACHABLE
192.168.1.8 dev wlp1s0 lladdr 44:08:0b:7e:68:22 REACHABLE
```

Whenever we want to send a packet to some IP address, we first lookup its MAC address in the ARP cache. If it is already in cache: fine, we can get it quickly! If there isn't we must generate ARP request to get it.

Example: When pc1 pings (ICMP request) s1 for the first time, we observe an ARP request and reply for each hop



Security Consideration - ARP/MAC Spoofing

In theory: MAC addresses are assigned by the manufacturer and are immutable.

In practice, **the linux command to change MAC address is**:

ip link set eth0 address 00:11:22:33:44:55

The problem is that some WiFi and Ethernet networks may use MAC-based authentication. Consequently, ARP alone is not secure. An attacker could "spoof" our MAC address and use it to authenticate.

Solution: network administrator should implement defenses against ARP attacks.

AC addresses are unique values for every network interface of any device → MAC addresses could be used to track a device through space Example: Wi-Fi based localization on smartphones 1. Devices send to their manufacturer the MAC addresses of near Wi-Fi access points 2. There exist databases1 2, even open ones, containing millions of MAC addresses with their position 3. By cross-referencing the MAC addresses sent by the device and the positions of known MAC addresses, you get a rough estimate of the device position

Security Consideration – Tracking and Localization

MAC addresses are unique values for every network interface of any device… So, MAC addresses could be used to track a device through space.

Example: Wi-Fi based localization on smartphones

1. Devices send to their manufacturer the MAC addresses of near Wi-Fi access points
2. There exist databases containing millions of MAC addresses with their position
3. By cross-referencing the MAC addresses sent by the device and the positions of known MAC addresses, you get a rough estimate of the device position

Researchers in have decoded the encrypted diagnostic traffic sent by Android and iOS devices, in particular the traffic exchanged when a device turns on the localization services For example, iPhones [1] with GPS localization enabled logs the MAC address of all nearby devices (connected to the same wifi) and send it to their servers with precise location.

IP Subnetting

An IPv4 address is a 4 bytes-long identifier, expressed in dotted notation, from 0.0.0.0 to 255.255.255.255

- example: 192.168.1.100

A subnet (or prefix) is a logical division of IP addresses. It is a pool of IP addresses. Examples:

- Subnet 192.168.1.0/24 - Contains addresses from 192.168.1.0 to 192.168.1.255
- Subnet 10.10.10.4/31 - Contains addresses from 10.10.10.4 to 10.10.10.5
- Subnet 10.50.0.0/16 - Contains addresses from 10.50.0.0 to 10.50.255.255

The number after the / is the netmask, the number of bits of the subnet

Which IP addresses belong to a subnet? An IP belongs to a subnet if its first `netmask` bits match the subnet.

1.	IP Address decimal	10	8	160	0
2.	Netmask /19	11111111	11111111	11100000	00000000
3.	IP Address binary	00001010	00001000	10100000	00000000

Fixed (Network ID) Variable (Host ID)

How many IP addresses belong to a subnet?

Example: consider the subnet of the previous slide 10.8.160.0/19. We have 19 fixed bits (network ID) and 13 variable bits (host ID). Number of addresses is 213

HOWEVER, we don't assign two addresses:

1. The lowest address 10.8.160.0 indicates the subnet itself
2. The highest address 10.8.191.255 is the broadcast address

So, the total number of assignable addresses is 213 - 2 = 8190

HOWEVER x2, by convention, all the 2 addresses of /31 subnets are assignable. E.g. in a subnet 10.0.0.0/31 both 10.0.0.0 and 10.0.0.1 are valid

To which subnet does an IP belong?

In other words, given an IP address and a netmask, how to find its subnet? You can obtain the subnet by applying the bitwise AND of the address and the netmask Example: considering an IP address 10.8.162.52 and knowing that its subnet has netmask /19, which is its subnet? Steps:

1.	IP Address decimal	10	8	162	52
2.	IP Address to binary	00001010	00001000	10100010	00110100
3.	Netmask* /19	11111111	11111111	11100000	00000000
4.	Subnet binary	00001010	00001000	10100000	00000000
5.	Subnet to decimal	10	8	160	0

IP Routing

Def. **Routing is the process of forwarding packets towards its final destination.**



The IP Subnets we have just seen are fundamental because they enable routing! The nodes doing routing are routers. For each incoming packet they look up in a Routing Table and decide which is the next hop (another router or the final node). Then, they simply send the packet on right interface to the next hop.

Routing Tables

We can get the routing table of a certain router using the command *ip route show*. Generally, each entry in the routing table indicates how to reach a certain subnet, not a specific IP address.

Destination Subnet	Interface	Metric
Default	eth0, via 10.50.0.1	100
10.2.128.0/18	wlp1s0	600
10.50.0.0/16	eth0	100
169.254.0.0/16	eth0	1000

Note that in the routing table, we store the interface to which forward a packet. But we did not specify an interface! next_hop_ip is itself routed recursively to find its target interface!

Forwarding Decision

When a router wants to send a message to the IP Address A, for each entry in the routing table, the router checks if A belongs in that entry. At the end:

1. **If only one subnet matches**: forward the packet there
2. **If more than one subnet matches**: Longest prefix match
3. **If no subnet matches**: Is there a “default” entry?
    - **Yes**: forward it using the default gateway
    - **No**: Network Unreachable error. Translates to “I don’t know how to reach that subnet”

Esempio: Supponiamo la seguente tabella di routing:

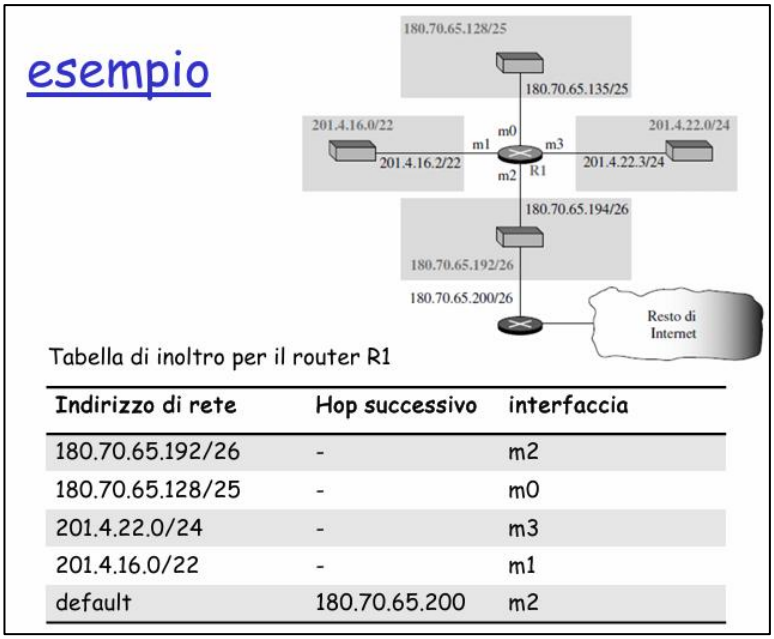


Tabella di inoltro per il router R1

Indirizzo di rete	Hop successivo	interfaccia
180.70.65.192/26	-	m2
180.70.65.128/25	-	m0
201.4.22.0/24	-	m3
201.4.16.0/22	-	m1
default	180.70.65.200	m2

Se dobbiamo inoltrare il datagramma, con indirizzo di destinazione 180.70.65.140 = 10110100 01000110 01000001 10001100, il router esegue i seguenti passaggi:

1. Confronta con la prima maschera (/26) ed avremo:

   10110100 01000110 01000001 10001100 (indirizzo destinazione)

   10110100 01000110 01000001 11000000 (maschera presente nella tabella)

   I due numeri non coincidono. Dobbiamo quindi controllare la entry successiva

2. Confrontiamo con la seconda maschera (/25) ed avremo:

   10110100 01000110 01000001 10001100 (indirizzo destinazione)

   10110100 01000110 01000001 10000000 (maschera presente nella tabella)

   I due numeri combaciano alla perfezione. Questa è la entry giusta. Possiamo quindi inoltrare il pacchetto all'interfaccia m0

Exercise: Address to route: **151.100.25.63** Routing table contains the following entries:

A) 151.64.0.0/10
B) 151.96.0.0/11
C) 151.0.0.0/8
D) 151.112.0.0/12
E) 151.104.25.0/28

Which one would you choose?

Exercise – unreachable_destinations

In this lab we try to observe the network behavior in case of errors. Depending on the nature of the error we observe a different behavior, useful when debugging. Launch unreachable_destinations lab, already configured, available now in the shared drive. Then:

1. From pc1 try to ping an unreachable local destination, e.g. 192.168.1.7, in the same subnet of pc1. Capture on eth0 pc1 (e.g., tcpdump -i et0 -w /shared/cap.pcap)
2. From pc1 try to ping an unreachable remote destination, e.g. 10.8.0.20, in the same subnet of s1. For this, capture packets on interface eth0 and eth1 of R2.
3. From pc1 try to ping a non-existent remote destination, e.g. 172.16.0.1. Capture on eth0 pc1
4. Observe with wireshark which ARP packets are exchanged in the various collision domains. Which errors do you observe?

Tip: You will need two terminals for pc1 (one pinging and another one sniffing). You can open a new terminal typing *kithara connect pc1*.